MACHINE LEARNING LAB · PROJECT REPORT

Multi-Cancer Detection System

Predicting cancer type, stage and survival from clinical patient records (TCGA)

TEST ACCURACY OF THE DEPLOYED MODELS

Organ	Stage	Survival
90.9%	51.8%	80.1%
23 classes · Random Forest baseline 11%	4 classes · Logistic Regression baseline 32%	2 classes · Logistic Regression baseline 71%

SUBMITTED BY
Pravesh Shrivastava
Aniket Nawkar

DATE
October 2026

Educational project. Predictions are statistical estimates from historical data and are not a medical diagnosis.

Contents

Abstract . . . 3
1. Introduction . . . 3
1.1 Problem statement . . . 3
1.2 Objectives . . . 3
1.3 Scope . . . 3
2. Dataset . . . 4
2.1 Columns . . . 5
3. Data Cleaning and Preparation . . . 5
3.1 Missing values . . . 5
3.2 Duplicates and outliers . . . 6
3.3 Exploratory analysis . . . 6
4. Methodology . . . 7
4.1 System design . . . 7
4.2 Target construction . . . 7
4.3 Feature selection and leakage control . . . 8
4.4 Preprocessing . . . 8
4.5 Models compared . . . 8
5. Results . . . 9
5.1 Organ prediction . . . 9
5.2 Stage prediction . . . 10
5.3 Survival prediction . . . 11
6. Web Application . . . 11
6.1 Request flow . . . 11
6.2 Example predictions . . . 11
6.3 Running the application . . . 12
7. Discussion and Limitations . . . 12
7.1 What worked . . . 12
7.2 Limitations . . . 12
7.3 Evaluation caveats . . . 13
8. Conclusion and Future Work . . . 13
Future work . . . 13
References . . . 14
Appendix A. Project Structure . . . 15
Appendix B. Technology Stack . . . 15

Abstract

This project builds a machine-learning system that answers three clinical questions about a patient who is known to have a tumour: **which organ** the cancer is in, **how advanced** it is (Stage I–IV), and **whether the patient is likely to survive**. We use 9,523 real, anonymised patient records from The Cancer Genome Atlas (TCGA), covering 32 cancer types grouped into 23 organs. After cleaning and encoding the data, six classifiers — Logistic Regression, K-Nearest Neighbours, Support Vector Classifier, Decision Tree, Random Forest and XGBoost — are trained and compared on each task. The three selected models are chained so that the predicted organ informs the stage model, and both feed the survival model, mirroring a clinical workflow.

On a held-out 20% test set the deployed models reach **90.9% accuracy for organ** (23 classes, macro F1 0.89), **51.8% for stage** (4 classes) and **80.1% for survival** (2 classes), against majority-class baselines of 11%, 32% and 71% respectively. Stage is the hardest task because the TNM fields that define stage must be withheld to prevent leakage. The models are served through a Flask web application that accepts partial patient information and returns all three predictions together with a risk-of-death percentage.

1. Introduction

1.1 Problem statement

Once a tumour has been found, three questions drive treatment planning and counselling: *Where is the cancer?*, *How far has it progressed?* and *What is the outlook?* Answering them normally requires specialist pathology and imaging review. This project investigates how much of that information can be estimated from routinely recorded clinical fields — age, sex, tumour histology code, staging fields, treatment and recurrence — using standard supervised learning.

1.2 Objectives

- Clean and explore a multi-cancer clinical dataset with substantial missing data.
- Train and compare six classification algorithms on each of three prediction tasks.
- Select features per task so that no model sees information that directly reveals its target (data leakage).
- Chain the three models into a single prediction pipeline and serve it through a web interface.
- Report results honestly against simple baselines and document the system's limitations.

1.3 Scope

Every patient in the dataset has cancer, so the system identifies *which* cancer a patient has — it does not screen for *whether* a person has cancer. It is an educational project and is not intended for clinical decision-making.

2. Dataset

The dataset combines patient identifiers from **TCGA-Reports** (Kefeli & Tatonetti, 2024) with clinical variables from the **TCGA PanCancer Atlas** obtained via cBioPortal, matched by patient ID. It is stored as a single CSV file with one row per patient.

Property	Value
Source	The Cancer Genome Atlas (TCGA) — TCGA-Reports + PanCancer Atlas clinical data
Patients (rows)	9,523 real, anonymised cancer patients; no duplicate rows
Columns	17 (2 numeric, 15 categorical)
Cancer types	32 TCGA cancer types, grouped into 23 organ classes
Age	14 – 90 years; mean 59.5, median 61, skew −0.44
Survival status	6,552 LIVING (70.6%) · 2,728 DECEASED (29.4%) · 243 missing

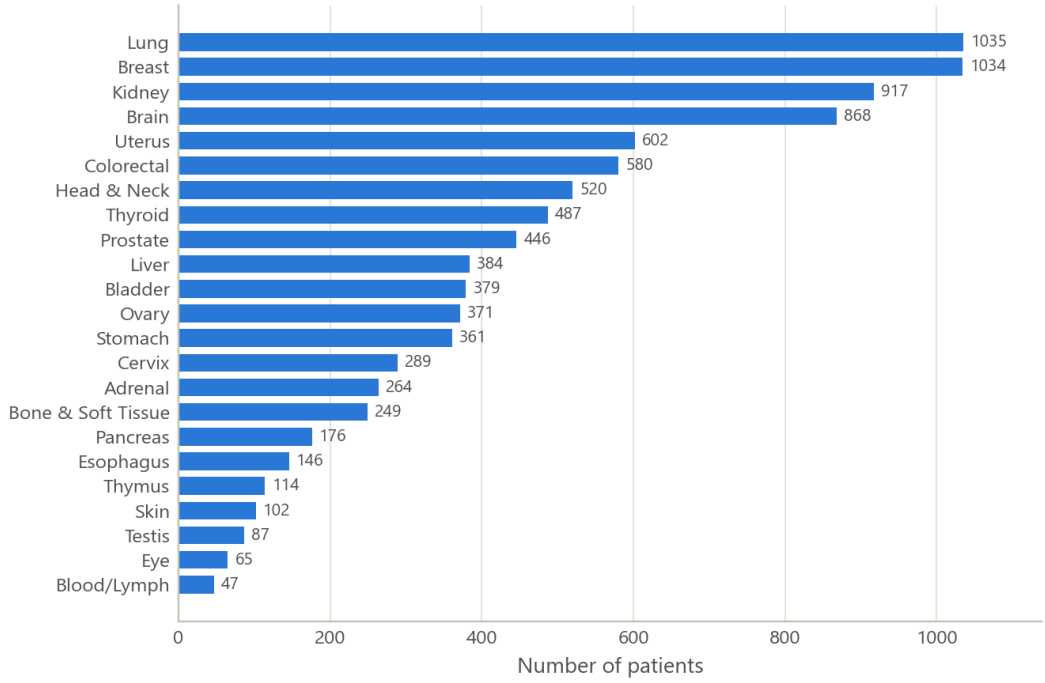


Figure 1. Patients per organ class. The classes are strongly imbalanced: Lung, Breast and Kidney each have over 900 patients, while Eye and Blood/Lymph have fewer than 70. Always predicting the largest class (Lung) would be correct only 10.9% of the time.

2.1 Columns

Column	Description
age	Age at diagnosis (years)
sex	Male / Female
race, ethnicity	As recorded by the hospital (not used as features — see §4.3)
histology_code	ICD-O-3 morphology code describing the tumour's cell type, e.g. 8500/3 (130 distinct values)
tumor_stage	Overall stage, Stage I–IV with sub-stages such as IIA, IIIB (29 distinct values)
t_stage, n_stage, m_stage	TNM staging — primary Tumour size, regional lymph Nodes, distant Metastasis
radiation_therapy	Whether the patient received radiation therapy
new_tumor_event	Whether the cancer returned after initial treatment
survival_status	LIVING / DECEASED at last follow-up (target of task 3)
survival_months	Months of follow-up (excluded: it leaks the survival outcome)
cancer_type, cancer_name	TCGA cancer code and full name (excluded: they reveal the organ)
organ	Grouped organ — Brain, Lung, Breast, … (target of task 1)

3. Data Cleaning and Preparation

3.1 Missing values

Missing data is the dataset's main quality problem: staging fields are absent for 29–38% of patients, largely because some cancers (e.g. brain tumours and sarcomas) are not staged with the Stage I–IV system at all. Rather than discarding these patients, we treat missingness as information.

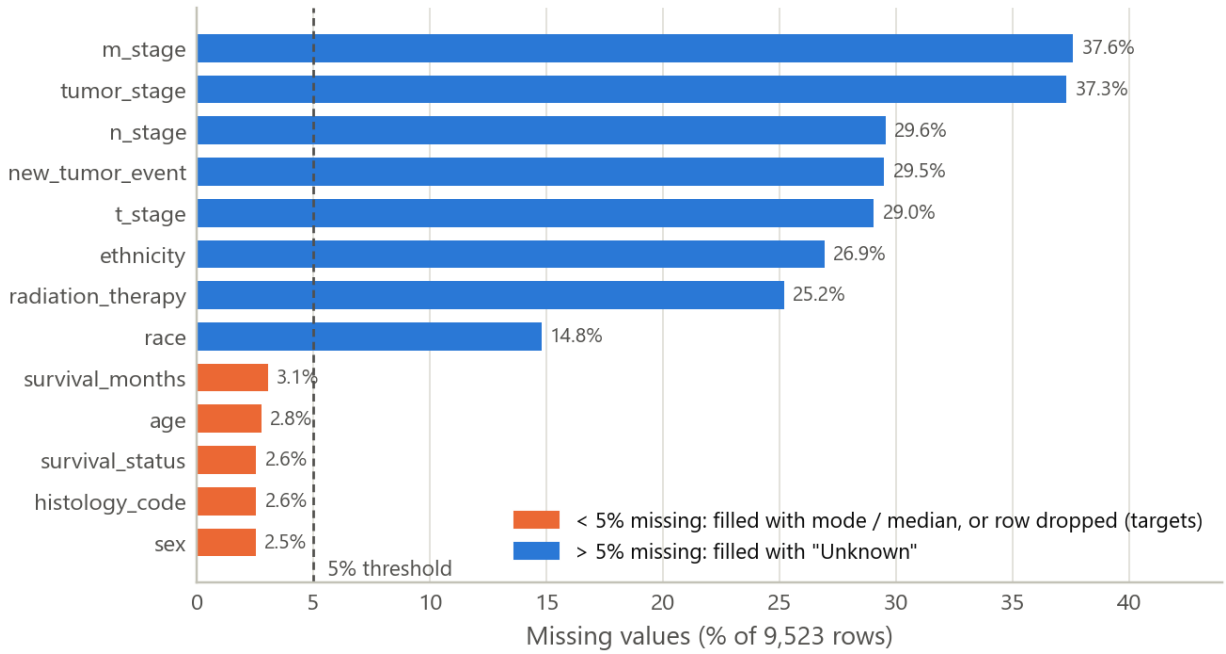


Figure 2. Share of missing values per column, with the 5% rule used to choose a filling strategy. Identifier and label columns (patient_id, cancer_type, cancer_name, organ) are complete and not shown.

Column type	Rule	Rationale
Numeric (age)	Fill with the median (61)	Distribution is mildly left-skewed (−0.44); the median is robust and a whole number.
Categorical, < 5% missing	Fill with the mode (sex, histology_code)	Too few gaps to justify a separate category.
Categorical, > 5% missing	Fill with the label "Unknown"	Lets the model learn from missingness — e.g. an unstaged tumour hints at brain or sarcoma.
Target columns	Never filled	Rows with a missing target are dropped for that task only.

3.2 Duplicates and outliers

No duplicate rows were found (every patient_id is unique). For outliers, the inter-quartile-range rule on *age* (Q1 = 51, Q3 = 70, lower fence 22.5 years) flags **101 patients**, all younger than 22.5. They were **kept**: they are genuine young patients with cancers that are common at younger ages (brain, thyroid, testicular), not data-entry errors, and removing them would bias the models against these cancers.

3.3 Exploratory analysis

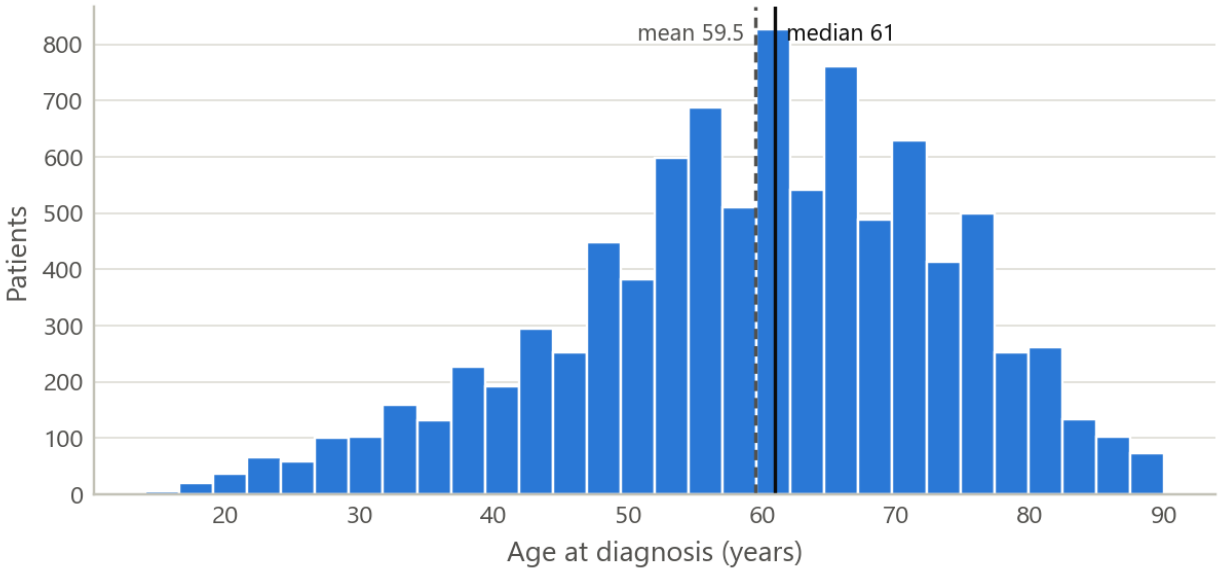


Figure 3. Age at diagnosis. Most patients are diagnosed between 50 and 75; the tail of younger patients is the group flagged by the IQR rule and deliberately kept.

Several organ classes are effectively single-sex (Ovary, Uterus and Cervix are female; Prostate and Testis are male; Breast is ~99% female), so *sex* is an informative feature for organ prediction. Mortality also differs sharply between cancers, which is why organ is used as an input to the survival model.

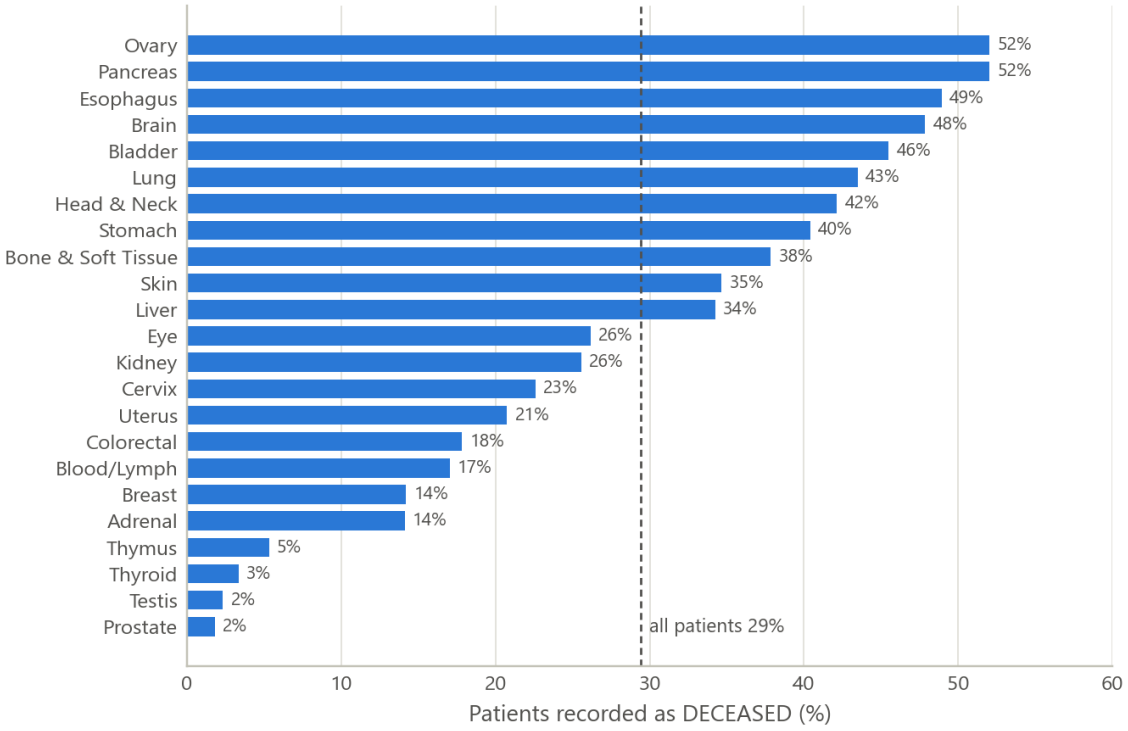


Figure 4. Share of patients recorded as deceased, by organ. Ovarian and pancreatic cancers exceed 50%, while prostate, testicular and thyroid cancers are below 5% — a 25-fold spread that the survival model can exploit once the organ is known.

4. Methodology

4.1 System design

Rather than one model with a combined output, the system trains a separate classifier for each question and chains them in the order information flows in a clinical workflow. The organ prediction is appended to the patient record before staging, and both organ and stage are available to the survival model.

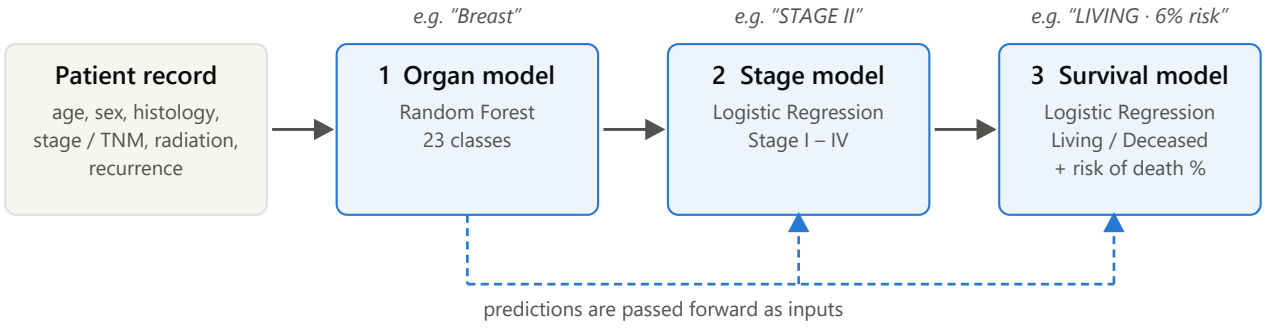


Figure 5. The three-model prediction chain used at inference time.

4.2 Target construction

- **Organ** — used directly (23 classes, no missing values; all 9,523 rows).
- **Stage** — sub-stages are collapsed to the four main stages with a regular expression (e.g. STAGE IIA → STAGE II). Patients with an Unknown or non-numeric stage are dropped, leaving 5,957 rows: Stage I 1,856 · II 1,884 · III 1,430 · IV 787.
- **Survival** — LIVING / DECEASED; the 243 rows without a status are dropped, leaving 9,280 rows.

4.3 Feature selection and leakage control

Each task receives only the inputs a clinician could plausibly have *before* knowing the answer. Columns that directly encode the target are removed:

Task	Target	Input features	Removed to avoid leakage
Organ	organ	age, sex, histology, tumour stage, T, N, M, radiation, recurrence	cancer_type, cancer_name, survival fields
Stage	stage (I–IV)	age, sex, histology, radiation, recurrence, **organ**	tumor_stage, T, N, M (they define the stage)
Survival	survival_status	age, sex, histology, tumour stage, T, N, M, radiation, recurrence, **organ**	survival_months

race and *ethnicity* were dropped from all three tasks because they did not improve accuracy, and excluding them avoids encoding demographic bias into the predictions.

4.4 Preprocessing

- **StandardScaler** on age (the only numeric input).
- **OneHotEncoder(handle_unknown='ignore')** on all categorical inputs, so an unseen category at prediction time is ignored rather than causing an error.
- **LabelEncoder** on the target, to convert class names to integers and back.
- Both transforms are combined in a **ColumnTransformer** fitted on the training split only, then applied unchanged to the test split and to new patients.
- **80 / 20 train–test split** with random_state = 42 (stratified by class for the survival task).

4.5 Models compared

Model	Configuration	Character
Logistic Regression	max_iter = 2000	Linear, fast, well-calibrated probabilities
K-Nearest Neighbours	default (k = 5)	Instance-based; sensitive to high-dimensional one-hot data
Support Vector Classifier	RBF kernel, default C	Non-linear margin classifier
Decision Tree	random_state = 42	Interpretable, prone to over-fitting
Random Forest	300 trees, random_state = 42	Bagged trees; robust to noise
XGBoost	default, random_state = 42	Gradient-boosted trees

All models are evaluated on the same held-out test set using accuracy, with per-class precision, recall and F1 for the selected model. Each selected model is saved with *joblib* together with its preprocessor and label encoder (three files per task, nine in total).

5. Results

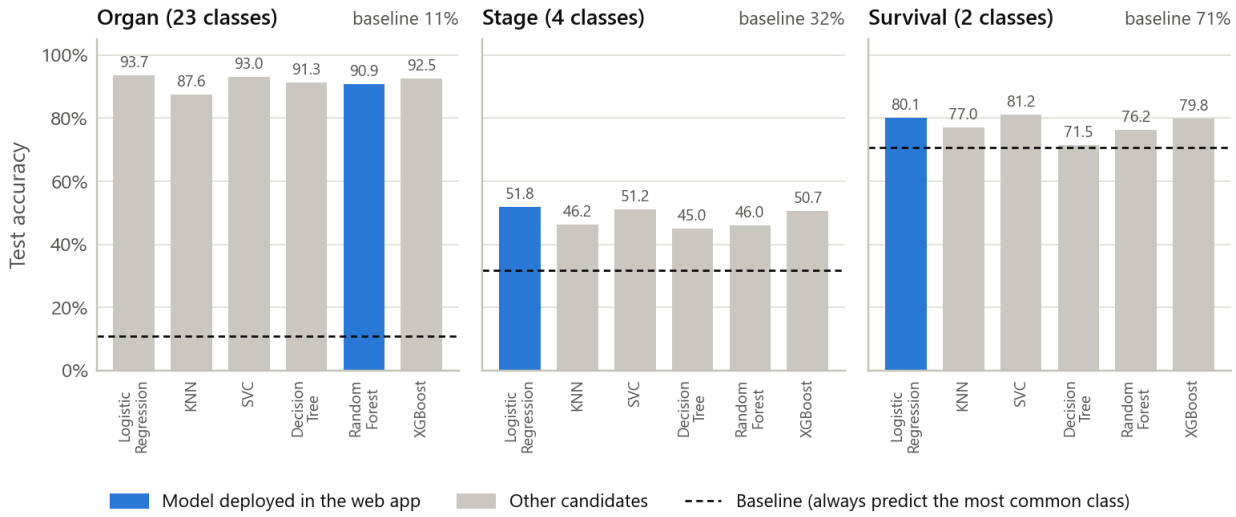


Figure 6. Test accuracy of all six models on each task. The dashed line is the majority-class baseline; the blue bar is the model deployed in the web application.

Model	Organ	Stage	Survival
Logistic Regression	**93.7%**	**51.8%** ●	80.1% ●
K-Nearest Neighbours	87.6%	46.2%	77.0%
Support Vector Classifier	93.0%	51.2%	**81.2%**
Decision Tree	91.3%	45.0%	71.5%
Random Forest	90.9% ●	46.0%	76.2%
XGBoost	92.5%	50.7%	79.8%
Majority-class baseline	*10.9%*	*31.6%*	*70.6%*
Test-set size	1,905	1,192	1,856

Bold marks the best accuracy per task; ● marks the deployed model.

5.1 Organ prediction

All six models are far above the 10.9% baseline; the linear models do best (Logistic Regression 93.7%, SVC 93.0%), which suggests the one-hot encoded histology code makes the classes close to linearly separable. The Random Forest saved for deployment scores 90.9% with a **macro F1 of 0.89** and weighted F1 of 0.91. Removing the histology code lowers organ accuracy from about 91% to about 73%, so it is the single most informative feature.

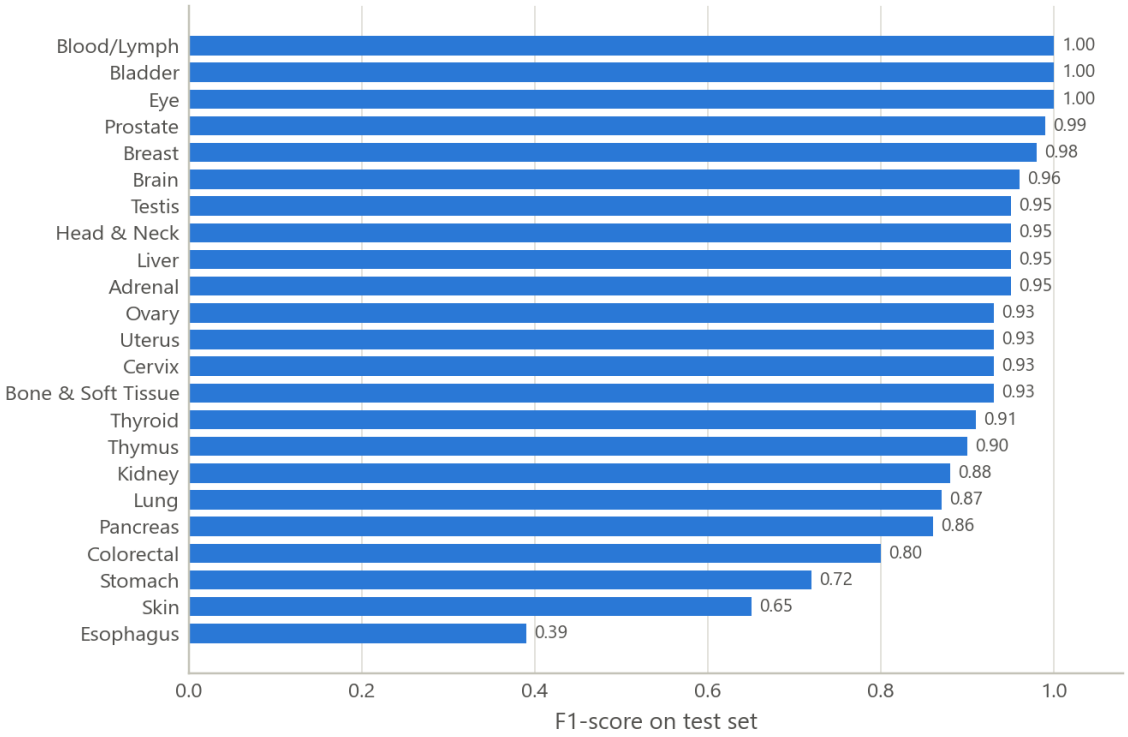


Figure 7. Per-class F1 of the deployed Random Forest on the organ test set (1,905 patients). Most classes score above 0.85; the weak classes are the small ones or those sharing histology with a neighbour.

The weakest classes are **Esophagus** (recall 0.28, 32 test patients), **Skin** (recall 0.50, 22) and **Stomach** (recall 0.73, 67). The code 8140/3 (adenocarcinoma, not otherwise specified) is the most common histology for both oesophageal (59%) and stomach (39%) cancer, and is also used for colorectal, lung and prostate tumours, so the model confuses these neighbouring organs; skin has very few test examples, so each error moves its recall substantially.

5.2 Stage prediction

Stage is the hardest task. Logistic Regression is best at **51.8%**, about 20 points above the 31.6% baseline. The limitation is by design: stage is *defined* by the T, N and M values, which must be removed to avoid leakage, leaving only indirect signals such as histology, recurrence and organ.

Class	Precision	Recall	F1	Support
Stage I	0.56	0.74	0.64	366
Stage II	0.51	0.56	0.53	371
Stage III	0.39	0.25	0.31	296
Stage IV	0.57	0.42	0.48	159
Macro average	0.51	0.49	0.49	1,192

Classification report for the deployed stage model (Logistic Regression).

Most errors fall between neighbouring stages (e.g. II vs III); Stage III is recalled only 25% of the time and is mostly absorbed into Stage II. In the web app this model is only used when the user does not enter a stage — if a stage is known it is passed straight through.

5.3 Survival prediction

SVC achieves the highest accuracy (81.2%), but Logistic Regression (80.1%) is deployed because it provides class probabilities via *predict_proba*, which the app needs to report a **risk-of-death percentage**. The SVC as trained (without probability=True) cannot do this; the trade-off costs 1.1 points of accuracy.

Class	Precision	Recall	F1	Support
DECEASED	0.69	0.58	0.63	546
LIVING	0.83	0.89	0.86	1,310
Macro average	0.76	0.73	0.75	1,856

Classification report for the deployed survival model (Logistic Regression).

Because the classes are imbalanced (71% living), accuracy alone flatters the model. The more important figure is **recall for DECEASED: 0.58** — the model identifies a little over half of the patients who died. Precision for DECEASED is 0.69, so most high-risk flags are correct.

6. Web Application

The trained models are served by a small **Flask** application (*app.py*) with a single page rendered from a Jinja2 template (*templates/index.html*) and styled by *styles/style.css*. At start-up it loads the nine saved files and reads the dataset once to populate the drop-down menus with every valid histology code and stage value, each preceded by **Unknown** as the default.

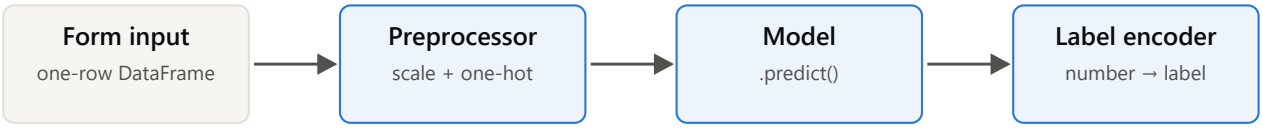


Figure 8. Each task's three saved files turn a form submission into a readable label.

6.1 Request flow

- The submitted form is converted into a one-row pandas DataFrame with the same columns used in training.
- The **organ** model predicts first, and the result is added to the record.
- **Stage**: if the organ is Brain or Bone & Soft Tissue the app reports *Not staged* (these cancers do not use Stage I–IV); if the user entered a stage it is shown as entered; otherwise the stage model predicts it and the prediction is passed on.
- The **survival** model predicts Living / Deceased and, from its class probabilities, the risk of death.

6.2 Example predictions

The four cases below were run through the saved models using the same logic as the web app; outputs are reproduced exactly.

Input	Case 1	Case 2	Case 3	Case 4
Age / sex	52 / F	58 / F	65 / M	60 / M
Histology code	8500/3	9401/3	8140/3	8140/3
Tumour stage	STAGE IIA	Unknown	STAGE IV	Unknown
T / N / M	T2 / N0 / M0	Unknown	T3 / N1 / M1	Unknown
Radiation / recurrence	Yes / No	Yes / Yes	No / No	No / No
Predicted organ	**Breast**	**Brain**	**Colorectal**	**Prostate**
Stage	IIA (entered)	Not staged	IV (entered)	**II (predicted)**
Survival · risk	Living · 4%	Deceased · 71%	Living · 31%	Living · 3%

The results are clinically sensible: a ductal carcinoma (8500/3) in a woman is classified as breast with low risk; an anaplastic astrocytoma (9401/3) with recurrence is classified as an unstaged brain tumour with high risk; and metastatic (M1) colorectal adenocarcinoma receives a markedly higher risk than the early-stage cases.

6.3 Running the application

```
pip install -r requirements.txt
python app.py
# open http://127.0.0.1:5000
```

The app must be launched from the project folder so the relative paths to *scripts/* and *datasets/* resolve, and scikit-learn should match the version used for training so the pickled files load correctly.

7. Discussion and Limitations

7.1 What worked

- **Treating missingness as a category.** Filling sparse fields with "Unknown" kept every patient and lets the model use the absence of staging as a clue.
- **Per-task leakage control.** Removing the fields that define each target keeps the reported accuracies honest, at the cost of a lower stage score.
- **Chaining.** Feeding the predicted organ forward gives the stage and survival models strong context — mortality varies 25-fold between organs (Figure 4).

7.2 Limitations

- **All patients have cancer.** The system identifies which cancer, not whether a person has cancer.
- **Heavy reliance on histology code.** It lifts organ accuracy from ~73% to ~91%; without it, performance drops sharply.
- **Stage without TNM is inherently limited**, as shown by the 51.8% ceiling across all six models.
- **Error propagation.** The stage and survival models were trained with the true organ but receive the *predicted* organ in the app, so an organ error carries forward.
- **Bone cancer is not truly represented** — the "Bone & Soft Tissue" class is TCGA sarcoma, mostly soft tissue.

- **Population.** TCGA data comes mainly from US hospitals; performance on other populations (e.g. Indian patients) is untested.
- **Follow-up length.** Some patients have short follow-up, so a LIVING label does not guarantee long-term survival.

7.3 Evaluation caveats

These points do not invalidate the results but should be considered when comparing models:

- Each model was evaluated on a **single 80/20 split**; differences of one or two points (e.g. SVC vs Logistic Regression on survival) are within the variation one would expect between splits.
- Only the survival split is stratified; the organ and stage splits are random, which matters most for the smallest organ classes.
- All models use **default hyper-parameters**; no grid search or cross-validation was performed.
- Median and mode imputation was computed on the full dataset before splitting, a minor form of leakage.
- For organ, Logistic Regression (93.7%) outperformed the deployed Random Forest (90.9%) on this split.

8. Conclusion and Future Work

The project shows that routinely recorded clinical fields carry enough signal to identify the organ of origin with about 91% accuracy across 23 classes, and to estimate survival with 80% accuracy and a usable risk score. Stage prediction without TNM data remains difficult at about 52%. Chaining three task-specific models and serving them through a simple web form produces an end-to-end tool that handles incomplete inputs gracefully.

Future work

- Use **stratified k-fold cross-validation** and **GridSearchCV** to tune each model and compare them with confidence intervals; revisit the organ model choice on that basis.
- Address class imbalance with class weights or resampling to raise recall for DECEASED and for small organ classes such as Esophagus and Skin.
- Train the stage and survival models on *predicted* organ (or out-of-fold predictions) to reduce error propagation in the chain.
- Model survival as **time-to-event** (e.g. Cox regression) using survival_months, instead of a binary label.
- Add explanations (feature importance or SHAP) to the web page so users can see why a prediction was made.
- Validate on data from other populations before any wider use.

References

Kefeli, J. & Tatonetti, N. (2024). TCGA-Reports: A machine-readable pathology report resource for benchmarking text-based AI models. *Patterns*.

The Cancer Genome Atlas (TCGA) PanCancer Atlas clinical data, accessed via cBioPortal for Cancer Genomics.

Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

Chen, T. & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of KDD 2016*.

Appendix A. Project Structure

Path	Purpose
app.py	Flask web application: loads models, handles the form, chains the three predictions
datasets/cancer_patients_tabular.csv	Source dataset (9,523 patients × 17 columns)
scripts/ML_project.ipynb	Data cleaning, exploratory analysis, model training and evaluation
scripts/organ_*.pkl	Organ task: model, preprocessor, label encoder
scripts/stage_*.pkl	Stage task: model, preprocessor, label encoder
scripts/survival_*.pkl	Survival task: model, preprocessor, label encoder
templates/index.html	Web page (Jinja2 template)
styles/style.css	Page styling
requirements.txt	Python dependencies for running the app

Appendix B. Technology Stack

Layer	Tools
Data and ML	Python, pandas, NumPy, scikit-learn, XGBoost
Visualisation	Matplotlib, Seaborn
Model storage	joblib
Web application	Flask, Jinja2, HTML, CSS (gunicorn for deployment)